Introduzione
alla compilazione automatica:
Make e Makefile.

Un codice sorgente

**Un codice scritto in un
qualsiasi linguaggio di programmazione ad alto livello
si compone di diverse "parti",
ognuna (preferibilmente) scritta in un file diverso.**

```
#include <stdio.h>
main() {
…
prog1(n,A);
prog2(A,B);
prog3(n,B);
}
```

Il programma chiamante

`main.c`

Un codice sorgente

**Un codice scritto in un
qualsiasi linguaggio di programmazione ad alto livello
si compone di diverse "parti",
ognuna (preferibilmente) scritta in un file diverso.**

```
#include <stdio.h>
main() {
…
prog1(n,A);
prog2(A,B);
prog3(n,B);
}
```

Le routine ausiliarie.

`prog1.c`

`prog2.c`

`prog3.c`

Generazione di un programma eseguibile

**La generazione di un programma eseguibile a partire
da un codice scritto in un linguaggio
di programmazione ad alto livello
si compone essenzialmente di due fasi:**

I FASE traduzione – Compilazione del programma

```
cc -c main.c prog1.c prog2.c prog3.c
```

…dai file.c ai file.o
cioè dai file sorgenti ai file oggetto

Generazione di un programma eseguibile

La generazione di un programma eseguibile a partire da un codice scritto in un linguaggio di programmazione ad alto livello si compone essenzialmente di due fasi:

II FASE linking - Creazione del programma eseguibile

```
cc -o ese.exe main.o prog1.o prog2.o prog3.o
```

...dai file.o al file.exe
cioè creazione del file eseguibile dall'unione di tutti gli oggetti

In generale

La compilazione di un codice può essere un'operazione molto laboriosa.

- Aggregazione di tutti i files sorgenti.
- Utilizzo di funzioni appartenenti ad opportune librerie.
- ...

Domanda

E' possibile rendere più semplici ed automatiche queste operazioni
?

Soluzione 1

E' possibile scrivere
uno *script*
che esegue sequenzialmente
tutte le operazioni necessarie!

Esempio di *script*

```
#! /bin/sh

cc –c main.c
cc –c prog1.c
cc –c prog2.c
cc –c prog3.c
cc –o prova.exe main.o prog1.c prog2.c prog3.c
```

compila.sh

```
Shell>  chmod +x compila.sh
Shell>  ./compila.sh
```


Soluzione 2

Utilizzare il programma *make* !

Il funzionamento del programma *make* esegue la compilazione in modo automatico rispettando queste due regole:

1. La **dipendenza** tra file
2. L'**ordine temporale** di creazione dei file sorgenti, oggetti ed eseguibili

Il programma MAKE.

Il programma make esegue sequenzialmente tutte le operazioni descritte in un file generalmente chiamato *makefile* o *Makefile.*

Un *makefile* è uno *script* specializzato per il programma make.
Contiene:
- **definizioni di macro**
- **descrizioni delle operazioni da eseguire.**

Il programma MAKE.

Il programma make esegue sequenzialmente tutte le operazioni descritte in un file generalmente chiamato *makefile* o *Makefile.*

Per eseguire il makefile digitare "make"

```
Shell>  make
```

Se il makefile ha un altro nome, es makeprog, digitare:

```
Shell>  make –f makeprog
```

Esempio

```
# makefile di un programma
# prima versione

ese.exe: main.o prog1.o prog2.o prog3.o
        cc -o prog main.o prog1.o prog2.o prog3.o

main.o : main.c
        cc -c main.c

prog1.o : prog1.c
        cc -c prog1.c

prog2.o : prog2.c
        cc -c prog2.c

prog3.o : prog3.c
        cc -c prog3.c
```


Esempio: linee di dipendenza

```
# makefile di un programma
# prima versione

ese.exe: main.o prog1.o prog2.o prog3.o
        cc -o prog main.o prog1.o prog2.o prog3.o

main.o : main.c
        cc -c main.c

prog1.o : prog1.c
        cc -c prog1.c

prog2.o : prog2.c
        cc -c prog2.c

prog3.o : prog3.c
        cc -c prog3.c
```

Linee di dipendenza

Makefile

Descrizione: linee di dipendenza

In un makefile si distinguono diversi tipi di linee:

- Linee di dipendenza

  Definisce una relazione tra un target e uno o più file detti di dipendenza.

  L'assegnazione di un target avviene con il carattere "**:**"

  `main.o : main.c`

Esempio: linee di comando

```
# makefile di un programma
# prima versione

ese.exe: main.o prog1.o prog2.o prog3.o
        cc -o ese.exe main.o prog1.o prog2.o prog3.o

main.o : main.c
        cc -c main.c

prog1.o : prog1.c
        cc -c prog1.c

prog2.o : prog2.c
        cc -c prog2.c

prog3.o : prog3.c
        cc -c prog3.c
```

Linee di comando

Makefile

Descrizione: linee di comando

In un makefile si distinguono diversi tipi di linee:

- Linee di comando

  Definisce l'operazione da effettuare sui file di dipendenza per ottenere il corrispondente target

  La linea di comando comincia dopo uno spazio di tabulazione

```
<TAB>cc -c main.c
```


Esempio: linee di commento

```
# makefile di un programma
# prima versione

ese.exe: main.o prog1.o prog2.o prog3.o
        cc -o ese.exe main.o prog1.o prog2.o prog3.o

main.o : main.c
        cc -c main.c

prog1.o : prog1.c
        cc -c prog1.c

prog2.o : prog2.c
        cc -c prog2.c

prog3.o : prog3.c
        cc -c prog3.c
```

Linee di commento

Makefile

Descrizione: linee di commento

In un makefile si distinguono diversi tipi di linee:

- Linee di commento

  Linee di testo per la descrizione delle operazioni, Cominciano col simbolo **#**

```
# questo è un commento
```


Esempio: linee per la definizione di macro

```
# makefile di un programma
# seconda versione

OBJ = main.o prog1.o prog2.o prog3.o

ese.exe: $(OBJ)
        cc -o ese.exe $(OBJ)

main.o : main.c
        cc -c main.c

prog1.o : prog1.c
        cc -c prog1.c

prog2.o : prog2.c
        cc -c prog2.c

prog3.o : prog3.c
        cc -c prog3.c
```

Definizione di una macro

Makefile

Descrizione: linee per la definizione di macro

In un makefile si distinguono diversi tipi di linee:

- Linee di definizione Macro

  Definisce ad es. un nome simbolico per un insieme di oggetti. La sintassi è la seguente:

  ```
  OBJ = prog1.o prog2.o prog3.o
  ```

  Il riferimento al nome simbolico si effettua tramite **$**

  ```
  ese.exe: $(OBJ)
        cc -o ese.exe $(OBJ)
  ```


Osservazione

Non è sempre necessario specificare in un Makefile tutti i target ed i comandi da eseguire.

Esistono delle regole predefinite che il make è in grado di riconoscere senza che esse siano specificate!

Esempio: Istruzioni di compilazione

```
# makefile di un programma
# terza versione

OBJ = main.o prog1.o prog2.o prog3.o

ese.exe: $(OBJ)
      cc -o ese.exe $(OBJ)

main.o : main.c

prog1.o : prog1.c

prog2.o : prog2.c

prog3.o : prog3.c
```

Il make riconosce i files con suffisso ".c" e fa eseguire se necessario la loro compilazione

Makefile

Osservazione

Il make ricorre alle regole predefinite
Solo se nel makefile una linea di dipendenza non è seguita da una linea di comando

Esecuzione di altri target

Il make esegue di default il primo target specificato nel makefile.
Per eseguire un altro target bisogna specificarlo
Subito dopo il comando make

Esempio: altri target

```
# makefile di un programma
# quarta versione

OBJ = main.o prog1.o prog2.o prog3.o

ese.exe: $(OBJ)
        cc -o ese.exe $(OBJ)

main.o : main.c

prog1.o : prog1.c
prog2.o : prog2.c
prog3.o : prog3.c

clean: $(OBJ) ese.exe
        rm -f $(OBJ) ese.exe
```

Makefile

Il target clean consente Di cancellare di tutti i .o E l'eseguibile

```
Shell>  make clean
```


Esempio: regole di Inferenza

```
# makefile di un programma
# quarta versione

OBJ = main.o prog1.o prog2.o prog3.o

ese.exe: $(OBJ)
        cc -o ese.exe $(OBJ)

main.o : main.c

.c.o :
        cc -c $*.c

clean: $(OBJ) prog
        rm -f $(OBJ) prog
```

Makefile

Questa regola definisce un'unica istruzione di compilazione per tutti i file .c

Regole di Inferenza

```
# makefile di un programma
# quarta versione

OBJ = main.o prog1.o prog2.o prog3.o

ese.exe: $(OBJ)
        cc -o ese.exe $(OBJ)

main.o : main.c

.c.o :
        cc -c $<

clean: $(OBJ) prog
        rm -f $(OBJ) prog
```

Makefile

Questa regola definisce un'unica istruzione di compilazione per tutti i file .c

```
Shell>  make clean
```

Regole di Inferenza

Le regole di inferenza permettono di semplificare e compattare le istruzioni nel makefile.

Esempio: Un makefile per un programma MPI

```
# Esempio: makefile (MakeMPI) per compilare un programma
con due codici sorgenti: main.c and mpihello.c

CC = mpicc
CFLAGS = -O3
LFLAGS = $(CFLAGS)
EXE = mpihello.exe
SRC = main.c mpihello.c

.SUFFIXES: .c.o

OBJ = $(SRC:.c=.o)

.c.o: $(CC) $(CFLAGS) -c $<

all: $(EXE)
$(EXE): $(OBJ)
        $(CC) $(LFLAGS) -o $@ $(OBJ)
clean:
        rm -f $(OBJ) $(EXE)
```

MakeMPI

FINE

Introduzione alla compilazione automatica: Make e Makefile.